

Рекурсия. Передача изменяемых и неизменяемых данных в функцию

1. Что такое рекурсия?

Рекурсия - это метод программирования, при котором функция **вызывает саму себя** до тех пор, пока не будет достигнут базовый случай.

Пример рекурсивного вызова функции

```
def countdown(n):  
    if n == 0: # Базовый случай  
        print("Finish!")  
    else:  
        print(n)  
        countdown(n - 1) # Рекурсивный вызов  
  
countdown(5)
```

Вывод:

```
5  
4  
3  
2  
1  
Finish!
```

Компоненты рекурсии

Любая рекурсивная функция должна содержать:

1. **Базовый случай** – условие выхода из рекурсии (иначе произойдёт бесконечный вызов).
2. **Рекурсивный вызов** – уменьшает размер задачи при каждом вызове.

2. Рекурсивные вычисления

Рекурсия часто используется для решения задач, которые можно разбить на более простые подзадачи.

Пример 1: Факториал числа (n!)

Факториал числа n - это произведение всех чисел от 1 до n:

$$n! = n * (n-1) * (n-2) * \dots * 1$$

```
def factorial(n):  
    if n == 1: # Базовый случай  
        return 1  
    return n * factorial(n - 1) # Рекурсивный вызов  
  
print(factorial(5)) # 120
```

Разбор вызовов:

```
factorial(5) = 5 * factorial(4)  
factorial(4) = 4 * factorial(3)  
factorial(3) = 3 * factorial(2)  
factorial(2) = 2 * factorial(1)  
factorial(1) = 1 (базовый случай)
```

Пример 2: Числа Фибоначчи

Числа Фибоначчи вычисляются по формуле:

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0, F(1) = 1$$

```
def fibonacci(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    return fibonacci(n - 1) + fibonacci(n - 2)  
  
print(fibonacci(6)) # 8
```

Когда использовать рекурсию?

- При решении задач, которые можно разделить на одинаковые подзадачи.
- При работе со сложными структурами данных (деревья, графы).
- Когда итеративное решение слишком громоздкое.

3. Изменяемые и неизменяемые объекты в Python

Все объекты в Python делятся на два типа:

1. Неизменяемые (immutable): int, float, str, tuple, bool.
2. Изменяемые (mutable): list, set, dict.

Передача неизменяемых объектов в функцию

При передаче неизменяемого объекта в функцию создаётся его копия, и изменения не затрагивают оригинальную переменную.

```
def modify_number(n):  
    n += 1  
    print("Внутри функции:", n)  
  
x = 5  
modify_number(x)  
print("После функции:", x)
```

Вывод:

```
Внутри функции: 6  
После функции: 5 # Оригинальная переменная не изменилась
```

Почему?

int - неизменяемый объект, поэтому в modify_number создаётся копия n, а не изменяется оригинальный x.

Передача изменяемых объектов в функцию

При передаче изменяемого объекта в функцию передаётся ссылка на оригинальный объект. Изменения в функции повлияют на исходные данные.

```
def modify_list(lst):  
    lst.append(100)  
    print("Внутри функции:", lst)  
  
nums = [1, 2, 3]  
modify_list(nums)  
print("После функции:", nums)
```

Вывод:

```
Внутри функции: [1, 2, 3, 100]  
После функции: [1, 2, 3, 100] # Оригинальный список изменился
```

Почему?

list - изменяемый объект, поэтому в modify_list изменения происходят в оригинальном списке.

4. Рекурсия с изменяемыми и неизменяемыми объектами

Пример 1: Изменяемый список в рекурсивной функции

```
def reverse_list(lst, index=0):  
    if index < len(lst) // 2:  
        lst[index], lst[-index-1] = lst[-index-1], lst[index]  
        reverse_list(lst, index + 1)  
  
numbers = [1, 2, 3, 4, 5]  
reverse_list(numbers)  
print(numbers) # [5, 4, 3, 2, 1]
```

Разбор:

- Список передаётся по ссылке, поэтому изменения затрагивают оригинальный объект.

- Рекурсивный вызов обменивает местами `lst[index]` и `lst[-index-1]`, пока `index` не достигнет середины.

Пример 2: Неизменяемый объект в рекурсии

```
def reverse_string(s):  
    if len(s) == 0:  
        return s  
    return s[-1] + reverse_string(s[:-1])  
  
print(reverse_string("hello")) # "olleh"
```

Разбор:

- Строка - неизменяемый объект.
- Каждый рекурсивный вызов создаёт новую строку, так как `s[:-1]` возвращает новый объект.

5. Когда использовать изменяемые и неизменяемые объекты в рекурсии?

1. Используйте изменяемые объекты, если нужно изменять оригинальный список или словарь.
2. Используйте неизменяемые объекты, если нужно создавать и возвращать новые значения.

Пример:

Рекурсивная функция, изменяющая переданный список (изменяемый объект):

```
def increment_list(lst, index=0):  
    if index < len(lst):  
        lst[index] += 1  
        increment_list(lst, index + 1)  
  
nums = [1, 2, 3]  
increment_list(nums)  
print(nums) # [2, 3, 4]
```

Рекурсивная функция, возвращающая новый список (**неизменяемый объект**):

```
def incremented_list(lst):
    if not lst:
        return []
    return [lst[0] + 1] + incremented_list(lst[1:])

nums = [1, 2, 3]
new_nums = incremented_list(nums)
print(new_nums)  # [2, 3, 4]
print(nums)      # [1, 2, 3] (не изменился!)
```

Вывод:

- Рекурсия полезна, когда задачу можно разбить на более мелкие подзадачи.
- Всегда важно определить базовый случай, чтобы избежать бесконечной рекурсии.
- Неизменяемые объекты передаются по значению и создают копии.
- Изменяемые объекты передаются по ссылке, изменяя оригинальные данные.